

✓ Algoritmos de optimización - Seminario

Nombre y Apellidos:

- Fabián Flores Vaca
- Pedro Borregón González

Url: <https://colab.research.google.com/drive/1rXFVBLk3tAdEFqE9F44Xee-uTx7JZTGG#scrollTo=hVbXYX-RfPWh>

Github: <https://github.com/pborregong/AlgoritmosOptimizacionFinal>

Problema:

1. Sesiones de doblaje
2. Organizar los horarios de partidos de La Liga
3. Combinar cifras y operaciones

Descripción del problema:

Problema 2. Organizar los horarios de partidos de La Liga(l)

- Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.
- Los horarios disponibles se conocen a priori y son los siguientes:

Día	Horarios disponibles
Viernes	20:00
Sábado	12:00, 16:00, 18:00, 20:00
Domingo	12:00, 16:00, 18:00, 20:00
Lunes	20:00

- En primer lugar se clasifican los equipos en tres categorías según el numero de seguidores(que tiene relación directa con la audiencia). Hay 3 equipos en la categoría A, 11 equipos de categoría B y 6 equipos de categoría C. • Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos)

Categoría del partido	Audiencia base
A-A	2 millones
A-B	1,3 millones
A-C	1 millón
B-B	0,9 millones
B-C	0,75 millones
C-C	0,47 millones

(No aparece explícitamente en la tabla del ejercicio, pero se deduce desde la tabla de ejemplo).

- Si el horario del partido no se realiza a las 20 horas del sábado se sabe que se reduce según los coeficientes de la siguiente tabla
- Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes.

Día	Hora	Coefficiente
Viernes	20:00	0,40
Sábado	12:00	0,55
Sábado	16:00	0,70
Sábado	18:00	0,80
Sábado	20:00	1,00
Domingo	12:00	0,45
Domingo	16:00	0,75
Domingo	18:00	0,85
Domingo	20:00	1,00
Lunes	20:00	0,40

- Es posible la coincidencia de horarios pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

Número de coincidencias	Coefficiente aplicado
0	1,00
1	0,75
2	0,55
3	0,40

Número de coincidencias	Coefficiente aplicado
4	0,30
5	0,25
6	0,22
7	0,20
8	0,20

La resolución del problema se ha orientado a realizar un algoritmo genético para su resolución.

(*) La respuesta es obligatoria

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

RESPUESTA

Posibilidades con restricciones

Hay que tener en cuenta que se calculan estas posibilidades dando por hecho que se parte de una jornada ya prefijada.

Para calcular las posibilidades con restricciones, primero nos fijamos cuál sería la posibilidad de que los 10 partidos diferentes puede elegir cualquiera de los 10 horarios diferentes, por lo que sería 10^{10} . Las restricciones es que hay que tener partido tanto el Viernes, como el Lunes. Si eliminamos el viernes cada partido solo podría elegir entre 9 horarios 9^{10} , pero aquí también se encuentran las que no utilizan el Lunes 8^{10} , es decir, no usan el Viernes pero si el Lunes: $9^{10} - 8^{10}$. De la misma manera los que no juegan el Lunes pero si el Viernes: $9^{10} - 8^{10}$. Nos queda que las que no usan ni el Viernes ni el Lunes son 8^{10} .

Por lo tanto:

$$10^{10} - [(9^{10} - 8^{10}) + (9^{10} - 8^{10}) + 8^{10}] = 4100173022$$

Posibilidades sin restricciones

$$10^{10} = 10000000000$$

```
p_sin = 10**10
viernes_no_lunes = 9**10 - 8**10
lunes_no_viernes = 9**10 - 8**10
sin_ambos = 8**10
total_a_restar = (viernes_no_lunes + lunes_no_viernes + sin_ambos)
total = p_sin - total_a_restar
print(f"Posibilidades con restricciones: {total}")
print(f"Posibilidades sin restricciones: {p_sin}")
```

```
Posibilidades con restricciones: 4100173022
Posibilidades sin restricciones: 10000000000
```

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

RESPUESTA

La estructura de datos que se usa en la resolución de este ejercicio es una lista de tuplas que representan al individuo (una jornada completa). Cada elemento de la lista es una tupla con el partido, que contiene otra tupla para mostrar el horario. (equipo1, equipo2), es una tupla que representa un partido. ((equipo1, equipo2), horario) es la tupla anterior junto con el horario. El individuo completo es una lista de estas tuplas. Se ha elegido esta estructura por lo sencillo que se hace realizar el cruce en estas estructuras en el algoritmo, ya que el cruce se realiza cogiendo un valor aleatorio de entre 3 y 7, y se resuelve de esta manera:

```
corte = random.randint(3,7)
hijo1 = individuo1[:corte] + individuo2[corte:]
```

Si hubieramos usado diccionarios, como los que se emplean para guardar el listado de equipos, las audiencias y las dos tablas de coeficientes, tendríamos que añadir un bucle para recorrer las claves y realizar el corte.

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

RESPUESTA

La solución es una jornada completa con los 10 partidos con su respectivo horario cada uno, y la función objetivo calcula la audiencia de esta jornada. Por cada uno de los 10 partidos, se calcula la audiencia de cada uno de la siguiente manera:

audiencia de cada partido = audiencia generica x coeficiente por horario x coeficiente de coincidencia

Posteriormente se suman todas las audiencias:

$$\text{audiencia total} = \text{audiencia}_1 + \text{audiencia}_2 + \dots + \text{audiencia}_{10}$$

Como la función mide lo bueno o malo que es nuestra solución aportada, y aquí se busca la mayor audiencia posible, corresponde a un problema de maximización.

DATOS PARA AMBOS ALGORITMOS

```
import random
equipos = {                                #la asignación de categorías ha sido por criterios personales
    "Athletic Club": "B",
    "Atlético de Madrid": "A",
    "CA Osasuna": "C",
    "Celta": "B",
    "Deportivo Alavés": "C",
    "Elche CF": "C",
    "FC Barcelona": "A",
    "Getafe CF": "B",
    "Levante UD": "C",
    "Málaga CF": "B",
    "Racing de Santander": "C",
    "Rayo Vallecano": "B",
    "RC Deportivo": "C",
    "RCD Espanyol": "B",
    "Real Betis": "B",
    "Real Madrid": "A",
    "Real Sociedad": "B",
    "Sevilla FC": "B",
    "Valencia CF": "B",
    "Villarreal CF": "B"
}
audiencias_mill = {
    "A-A": 2.0,
    "A-B": 1.3,
    "A-C": 1.0,
    "B-B": 0.9,
    "B-C": 0.75,
    "C-C": 0.47
}
c_horarios = {
    "V20": 0.4,
    "S12": 0.55,
    "S16": 0.7,
    "S18": 0.8,
    "S20": 1.0,
    "D12": 0.45,
    "D16": 0.75,
    "D18": 0.85,
    "D20": 1.0,
    "L20": 0.4
}
c_coincidencia = {
    1: 1.0,
    2: 0.75,
    3: 0.55,
    4: 0.40,
    5: 0.30,
    6: 0.25,
    7: 0.22,
    8: 0.20,
    9: 0.20
}
jornada_prefijada = [
    ("Real Madrid", "Celta"),
    ("FC Barcelona", "Valencia CF"),
    ("Atlético de Madrid", "Athletic Club"),
    ("Real Betis", "Sevilla FC"),
    ("Real Sociedad", "Getafe CF"),
    ("Villarreal CF", "Rayo Vallecano"),
    ("RCD Espanyol", "RC Deportivo"),
    ("CA Osasuna", "Elche CF"),
    ("Deportivo Alavés", "Levante UD"),
    ("Málaga CF", "Racing de Santander")
]
```

FUNCIÓN evaluar_audiencia PARA AMBOS ALGORITMOS

```
def evaluar_audiencia(poblacion):
    evaluaciones = []
    for individuo in poblacion:
        audiencia_total = 0
        audiencia_partidos = []
        repetidos_horarios = {} #contamos cuantas veces aparece cada horario, en un diccionario

        for partido, horario in individuo:
            equipo1, equipo2 = partido #separamos la tupla formada por cada equipo, pasando cada uno a una variable para buscar s
            categoria_equipo_1 = equipos[equipo1]
            categoria_equipo_2 = equipos[equipo2]
            if(categoria_equipo_1) > (categoria_equipo_2):
                categoria_por_partido = (categoria_equipo_2 + "-" + categoria_equipo_1)
            else:
                categoria_por_partido = (categoria_equipo_1 + "-" + categoria_equipo_2)

            repetidos_horarios[horario] = repetidos_horarios.get(horario, 0) + 1
            audiencia_sin_correcion = audiencias_mill[categoria_por_partido]
            coeficiente_horario = c_horarios[horario]
            audiencia_correcion_horario = audiencia_sin_correcion * coeficiente_horario
            audiencia_partidos.append({
                "partido": partido,
                "horario": horario,
                "categoria": categoria_por_partido,
                "audiencia": audiencia_correcion_horario
            })

        for valor in audiencia_partidos:
            horario = valor["horario"]
            audiencia = valor["audiencia"]
            c_repetido = c_coincidencia[repetidos_horarios[horario]]
            audiencia_final = audiencia * c_repetido
            valor["repeticiones"] = repetidos_horarios[horario]
            valor["audiencia_final"] = audiencia_final
            audiencia_total += audiencia_final

        evaluaciones.append((individuo, audiencia_total))
        #Este sería nuestro fitness, la audiencia total es el fitness de cada individuo de la población

    return evaluaciones
```

Diseña un algoritmo para resolver el problema por fuerza bruta

SOLUCIÓN

Para resolver el algoritmo por fuerza bruta se va usar <https://docs.python.org/es/3/library/itertools.html>, en concreto la función *product* y cuya documentación específica: "producto cartesiano, equivalente a un bucle for anidado", ideal para resolver este algoritmo por fuerza bruta y evitar poner 10 bucles anidados, que sería la otra opción.

```
from itertools import product
max_audiencia = 0
mejor_individuo = []
contador = 0 #ponemos un contador porque viendo las combinaciones que nos han salido al calucrlarlas, es inviable que se ej
max_repas = 50000 #ponemos 50000 repeticiones como máximo para que pare en algún momento
for x in product(c_horarios.keys(), repeat = 10):
    contador += 1
    if contador > max_repas: #Por si da algún resultado invalido, sin L o V
        break
    if "V20" not in x or "L20" not in x:
        continue
    #Como la función evaluar_audiencia se concibio para el algoritmo de optimización y posteriormente se desarrollo este, le
    #cada individuo en una lista, y así sería una población de solo un individuo y nos permite compararlo
    individuo = list(zip(jornada_prefijada, x)) #juntamos en una lista los partidos, junto con el horario
    evaluacion = evaluar_audiencia([individuo]) #le pasamos al individuo dentro de otra lista, como hemos comentado
    audiencia = evaluacion[0][1] #cogemos del primer elemento de la lista, en este caso el único, el valor de audiencia que t
    if audiencia > max_audiencia:
        max_audiencia = audiencia
        mejor_individuo = individuo

print(f"Mejor individuo: {mejor_individuo} con un máximo de audiencia de {max_audiencia}")

Mejor individuo: (((('Real Madrid', 'Celta'), 'V20'), (('FC Barcelona', 'Valencia CF'), 'V20'), (('Atlético de Madrid', 'Athl
```

Para ver como avanza, vemos que con **50000** combinaciones nos da lo siguiente como mejor individuo:

[('Real Madrid', 'Celta', 'V20'), ('FC Barcelona', 'Valencia CF', 'V20'), ('Atlético de Madrid', 'Athletic Club', 'V20'), ('Real Betis', 'Sevilla FC', 'V20'), ('Real Sociedad', 'Getafe CF', 'V20'), ('Villarreal CF', 'Rayo Vallecano', 'S20'), ('RCD Espanyol', 'RC Deportivo'), 'D18'), ('CA Osasuna', 'Elche CF', 'S18'), ('Deportivo Alavés', 'Levante UD', 'L20'), ('Málaga CF', 'Racing de Santander'), 'D20')] con un máximo de audiencia de **3.5355** millones de espectadores

En apenas unos segundos en google colab **200000**:

[('Real Madrid', 'Celta', 'V20'), ('FC Barcelona', 'Valencia CF', 'V20'), ('Atlético de Madrid', 'Athletic Club', 'V20'), ('Real Betis', 'Sevilla FC', 'V20'), ('Real Sociedad', 'Getafe CF', 'S12'), ('Villarreal CF', 'Rayo Vallecano', 'S20'), ('RCD Espanyol', 'RC Deportivo'), 'D18'), ('CA Osasuna', 'Elche CF', 'S18'), ('Deportivo Alavés', 'Levante UD', 'L20'), ('Málaga CF', 'Racing de Santander'), 'D20')]* con un máximo de audiencia de **4.1145 millones de espectadores

En un tiempo de 1 min. aproximado en google colab **20.000.000**:

[('Real Madrid', 'Celta', 'V20'), ('FC Barcelona', 'Valencia CF', 'V20'), ('Atlético de Madrid', 'Athletic Club', 'S12'), ('Real Betis', 'Sevilla FC', 'S20'), ('Real Sociedad', 'Getafe CF', 'D18'), ('Villarreal CF', 'Rayo Vallecano'), 'D20'), ('RCD Espanyol', 'RC Deportivo'), 'S18'), ('CA Osasuna', 'Elche CF', 'S16'), ('Deportivo Alavés', 'Levante UD'), 'L20'), ('Málaga CF', 'Racing de Santander'), 'D16')] con un máximo de audiencia de **5.7395** millones de espectadores.

Cabe destacar que esta solución que nos ha llevado 1 minuto aprox (google colab no me muestra los segundos). Haber probado 20 millones de combinaciones, supone $\frac{20.000.000}{10^{10}} \cdot 100 = 0,2\%$.

Si además contamos el tiempo de procesamiento, y considerandolo constante por una simple regla de 3:

1 — 0,2

x — 100

$\$x = 500$

Es decir, 500 minutos o 8,33 horas.

Estas soluciones podrían dar otros valores con otra jornada diferente a la elegida como valores iniciales.

Calcula la complejidad del algoritmo por fuerza bruta

RESPUESTA

La complejidad del algoritmo depende del número de combinaciones que generan los bucles anidados que tenemos, aunque les recorramos con product, por lo que podemos diferenciar:

- Sabemos que serían 10 bucles anidados, que con product es un tamaño de 10.
- Dentro de cada ejecución del bucle hacemos asignaciones a los partidos, es decir sería multiplicar la complejidad por 10, que son 10 partidos diferentes, pero esto se considera una constante.

Por lo tanto la complejidad de fuerza bruta sería: $10 \cdot N^{10}$, es decir complejidad de $O(N^{10})$

(*)Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

RESPUESTA

Para la solución de este apartado, vamos a usar un algoritmo genético, en el que hemos decidido las siguientes cuestiones:

- individuo: es una jornada de liga completa, no es lo mismo que un partido.
- población: serían varias propuestas de la misma jornada, pero en referente al horario, no en cuanto a los enfrentamientos que son los mismos para todos los individuos de la población.
- gen: sería un partido con un horario asignado.

El algoritmo básicamente lo que hace basado en una población elegida de 200 individuos, primero se genera esa población.

Posteriormente cogemos un grupo de 3 individuos, de los cuales cogemos el mejor y lo consideramos padre, y mismo proceso para la madre, eso sí evitando que pudieran ser el mismo individuo.

Una vez que hemos cruzado a ambos padres, siempre dan dos hijos, estos hijos tienen una parte del padre de entre el 30% y el 70%, y el resto de la madre. Cada hijo tiene una probabilidad de mutar, del 10%, y esta mutación se produce en solamente un gen, es decir en un partido asignado. Cuando sabemos si el hijo ha mutado o no, se evalúa que cumpla los requisitos de que se juegue un partido tanto el Viernes como el Lunes, sino es así se asigna un partido aleatorio a esos días, y si de por sí el hijo ya tenía más de uno esos días, se mantiene (es posible que todos ellos tengan varios partidos en la misma franja horaria). La mutación se agrega para evitar posibles estancamientos en máximos locales, ya que introduce mayor diversidad en la población.

Una vez que hemos generado hijos, vamos rellenando la población de 200 individuos definida. El mejor de cada generación pasa si o si a la siguiente, aunque sea padre, es decir se usa elitismo. Si por ejemplo como se generan 2 hijos en una población de 200, y ya

tenemos un individuo de la población anterior(el mejor) lo que se hace para rellenar el último hueco en 199 y que sobraría un hijo, se generan ambos hijos y se coge el mejor de ellos.

Como criterio de parada hemos establecido dos, es decir probar 1000 generaciones aunque se han hecho pruebas con muchas diferentes, y como criterio extra es que haya multiples generaciones sin mejorar, en este caso fijada como 500, en cuyo caso si tras 500 generaciones y no habiendo llegado a 1000 no se ha mejorado el algoritmo abandona. Este criterio de parada es el que realmente sucede.

FUNCIONES PROPIAS DEL ALGORITMO GENÉTICO

```
def comprobar_hijos_validos(hijo):
    numero_viernes = 0
    numero_lunes = 0
    indice_viernes = None
    indice_lunes = None
    for i, (partido, horario) in enumerate(hijo):
        if horario == "V20":
            numero_viernes +=1
            indice_viernes = i
        elif horario == "L20":
            numero_lunes +=1
            indice_lunes = i

    if numero_viernes == 0: #Miramos el numero de viernes
        indice_nuevo_viernes = random.randrange(len(hijo)) #cogemos un indice aleatorio para poner el partido de viernes porque
        if numero_lunes == 1: #revisamos, si el número de lunes es 1 para no tocarlo, si hay mas da igual sustituirle, y si ha
            while indice_nuevo_viernes == indice_lunes: #mientras que sea igual el indice, seguimos generando otro indice.
                indice_nuevo_viernes = random.randrange(len(hijo))

        partido = hijo[indice_nuevo_viernes][0]
        hijo[indice_nuevo_viernes] = (partido, "V20") #guardamos el nuevo horario
        indice_viernes = indice_nuevo_viernes #guardamos en indice_viernes el que acabamos de meter, por si acaso no había ning
        numero_viernes +=1

    if numero_lunes == 0:
        indice_nuevo_lunes = random.randrange(len(hijo))
        if numero_viernes == 1:
            while indice_nuevo_lunes == indice_viernes:
                indice_nuevo_lunes = random.randrange(len(hijo))

        partido = hijo[indice_nuevo_lunes][0]
        hijo[indice_nuevo_lunes] = (partido, "L20")
        #numero_lunes += 1
        #indice_lunes = indice_nuevo_lunes
    return hijo

def cruzar_descendencia(individuo1, individuo2):
    corte = random.randint(3,7) #Para no coger siempre la mitad de cada individuo, generamos un numero aleatorio entre 3-7, a
    hijo1 = individuo1[:corte] + individuo2[corte:]
    hijo2 = individuo2[:corte] + individuo1[corte:]
    hijo1 = comprobar_hijos_validos(hijo1)
    hijo2 = comprobar_hijos_validos(hijo2)
    return hijo1, hijo2

def mutar(hijo, horarios): #En la mutación cambiamos solamente un partido
    indice = random.randrange(len(hijo)) #generamos un número aleatorio
    partido = hijo[indice][0] #cogemos el partido que pertenece a ese indice
    horario_actual = hijo[indice][1] #cogemos el horario de ese partido
    horas_posibles = [ horario for horario in horarios if horario != horario_actual] #a traves de list comprehension elegimos
    nuevo_horario = random.choice(horas_posibles) #elegimos una opción aleatoria entre las horas posibles
    hijo[indice] = (partido, nuevo_horario)
    hijo = comprobar_hijos_validos(hijo)
    return hijo

def seleccion_torneo(poblacion_anterior, tamaño):
    posibles = random.sample(poblacion_anterior,tamaño)
    ganador = max(posibles, key = lambda x: x[1])
    return ganador

def crear_individuo_aleatorio(jornada, horarios):
    partidos = []
    partido_viernes, partido_lunes = random.sample(range(len(jornada)),2) #coge 2 partidos al azar DISTINTOS, que serán el de
    for i, partido in enumerate(jornada):
        #por cada elemento en jornada que hemos generado, comprobamos si es el indice aleatorio que nos ha tocado para lunes/vi
        if i == partido_lunes:
            hora = "L20"
        elif i == partido_viernes:
            hora = "V20"
```

```

else:
    hora = random.choice(horarios)

partidos.append((partido, hora))

return partidos

```

ALGORITMO GENÉTICO

```

def algoritmo_genetico(mejor, poblacion_anterior):
    nueva_poblacion = [mejor[0]] # mejor es individuo, puntuacion, asi que guardamos solo el individuo, la puntuación como te
    #lo primero de todo traemos al mejor de la poblacion generada aleatoriamente, como parte de usar el criterio de ELITISMO
    #Comenzaremos con lo definido com oturneo, donde se elegira poblacion al azar y de estos se seleccionara el mejor para re
    #La idea es quedarnos ocn el mejor de la poblacion anterior, y la descendencia de la poblacion_anterior hasta hacer otra
    while len(nueva_poblacion) < len(poblacion_anterior):
        mejor_seleccion_padre = seleccion_torneo(poblacion_anterior, 3) #llamamos a la función de selección por torneo, elegim
        mejor_seleccion_madre = seleccion_torneo(poblacion_anterior, 3)
        while mejor_seleccion_madre[0] == mejor_seleccion_padre[0]: #while por si llegamos a elegir a los dos iguales, entonces
            mejor_seleccion_madre = seleccion_torneo(poblacion_anterior, 3)
        hijo1, hijo2 = cruzar_descendencia(mejor_seleccion_padre[0], mejor_seleccion_madre[0])
        #Aquí introducimos las mutaciones, porque hemos comprobado que en 50 rondas o generaciones se queda atascada a partir c
        if random.random() < 0.1:
            hijo1 = mutar(hijo1, horarios)
        if random.random() < 0.1:
            hijo2 = mutar(hijo2, horarios)

        #Como estamos siendo elitistas, y guardamos el mejor de la poblacion anterior, y si queremos respetar el tamaño de la
        #va a quedar unhijo fuera, entonces controlamos este punto, evaluamos los hijos, comparamos cual es mejor de los 2 últi
        if len(nueva_poblacion) == len(poblacion_anterior) - 1:
            poblacion_hijos = [hijo1, hijo2]
            evaluacion_hijos = evaluar_audiencia(poblacion_hijos)
            mejor_hijo = max(evaluacion_hijos, key=lambda x: x[1])
            nueva_poblacion.append(mejor_hijo[0])
        else:
            nueva_poblacion.append(hijo1)
            nueva_poblacion.append(hijo2)

    return nueva_poblacion

#/////////////////////////# EJECUCIÓN DEL ALGORITMO GENÉTICO#/////////////////////////#
poblacion = []
horarios = list(c_horarios.keys())
#creamos una muestra de 200 individuos aleatorios completamente, aunque más adelante pida que genere entradas aleatorias, e
#de crear_individuo_aleatorio y recibe la jornada que es la que generaremos aleatoriamente en el apartado de más adelante,
for x in range(200):
    individuo = crear_individuo_aleatorio(jornada_prefijada, horarios)
    poblacion.append(individuo)

evaluacion = evaluar_audiencia(poblacion)
mejor = max(evaluacion, key = lambda x: x[1])

#print(mejor)
#algoritmo_genetico(mejor, evaluacion)
max_generaciones = 1000
limite_sin_mejora = 500
num_generaciones_sin_mejora = 0
mejor_generacion = 0

poblacion_actual = evaluacion #la población actual es la que hemos recibido la generación aleatoria, elegimos al mejor de e
mejor_poblacion_actual = mejor #mejor es el valor que traíamos del mejor de la generacion generada random
mejor_final = mejor

for generacion in range(max_generaciones):
    nueva_poblacion = algoritmo_genetico(mejor_poblacion_actual, poblacion_actual)
    poblacion_actual = evaluar_audiencia(nueva_poblacion)
    mejor_poblacion_actual = max(poblacion_actual, key=lambda x: x[1])

    if mejor_poblacion_actual[1] > mejor_final[1]:
        mejor_final = mejor_poblacion_actual
        num_generaciones_sin_mejora = 0
        mejor_generacion = generacion + 1 #guardamos como mejor generacion la actual, que corresponde con el lugar que ocupa el
    else:
        num_generaciones_sin_mejora += 1

    print(f"Generación {generacion + 1} -> con la audiencia de {mejor_poblacion_actual[1]:.3f}")

if num_generaciones_sin_mejora >= limite_sin_mejora:
    print(f"No hay mejora en {limite_sin_mejora} generaciones, por lo que se para de ejecutar")
    break

```

```

print(f"La mejor audiencia es: {mejor_final[1]}, en la generación {mejor_generacion}")
print(f"La mejor distribución es:")
for partido, horario in mejor_final[0]:
    print(partido, "--", horario)

```

```

Generación 471 -> con la audiencia de 6.756
Generación 472 -> con la audiencia de 6.756
Generación 473 -> con la audiencia de 6.756
Generación 474 -> con la audiencia de 6.756
Generación 475 -> con la audiencia de 6.756
Generación 476 -> con la audiencia de 6.756
Generación 477 -> con la audiencia de 6.756
Generación 478 -> con la audiencia de 6.756
Generación 479 -> con la audiencia de 6.756
Generación 480 -> con la audiencia de 6.756
Generación 481 -> con la audiencia de 6.756
Generación 482 -> con la audiencia de 6.756
Generación 483 -> con la audiencia de 6.756
Generación 484 -> con la audiencia de 6.756
Generación 485 -> con la audiencia de 6.756
Generación 486 -> con la audiencia de 6.756
Generación 487 -> con la audiencia de 6.756
Generación 488 -> con la audiencia de 6.756
Generación 489 -> con la audiencia de 6.756
Generación 490 -> con la audiencia de 6.756
Generación 491 -> con la audiencia de 6.756
Generación 492 -> con la audiencia de 6.756
Generación 493 -> con la audiencia de 6.756
Generación 494 -> con la audiencia de 6.756
Generación 495 -> con la audiencia de 6.756
Generación 496 -> con la audiencia de 6.756
Generación 497 -> con la audiencia de 6.756
Generación 498 -> con la audiencia de 6.756
Generación 499 -> con la audiencia de 6.756
Generación 500 -> con la audiencia de 6.756
Generación 501 -> con la audiencia de 6.756
Generación 502 -> con la audiencia de 6.756
Generación 503 -> con la audiencia de 6.756
Generación 504 -> con la audiencia de 6.756
Generación 505 -> con la audiencia de 6.756
Generación 506 -> con la audiencia de 6.756
Generación 507 -> con la audiencia de 6.756
Generación 508 -> con la audiencia de 6.756
Generación 509 -> con la audiencia de 6.756
Generación 510 -> con la audiencia de 6.756
Generación 511 -> con la audiencia de 6.756
Generación 512 -> con la audiencia de 6.756
Generación 513 -> con la audiencia de 6.756
Generación 514 -> con la audiencia de 6.756
Generación 515 -> con la audiencia de 6.756
No hay mejora en 500 generaciones, por lo que se para de ejecutar
La mejor audiencia es: 6.756, en la generación 15
La mejor distribución es:
('Real Madrid', 'Celta') -- D20
('FC Barcelona', 'Valencia CF') -- D16
('Atlético de Madrid', 'Athletic Club') -- D18
('Real Betis', 'Sevilla FC') -- S16
('Real Sociedad', 'Getafe CF') -- S18
('Villarreal CF', 'Rayo Vallecano') -- S20
('RCD Espanyol', 'RC Deportivo') -- D12
('CA Osasuna', 'Elche CF') -- L20
('Deportivo Alavés', 'Levante UD') -- V20
('Málaga CF', 'Racing de Santander') -- S12

```

(*)Calcula la complejidad del algoritmo

RESPUESTA

Dentro de cada generación se procesan P individuos (la población). Para cada uno de los individuos además se procesan los N partidos para poder calcular la audiencia. Por tanto, la complejidad de los algoritmos genéticos depende del número de generaciones, del tamaño de la población P y del número de partidos N , $O(G \cdot P \cdot N)$. En este caso se podrían considerar una constante $N = 10$ se podría decir que sería $O(G \cdot P)$

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

RESPUESTA

Como la generación de la población es por diseño aleatoria, incluimos aquí parte del código para generar los cruces o los partidos de manera aleatoria.

```

def entrada_random():
    #random.seed(3) #No usamos la semilla random

```



```

#Mezclamos los equipos de manera random, y mediante un bucle los elegimos de 2 en 2, para generar una primera jornada aleatoria
lista_equipos = list(equipos.keys())
random.shuffle(lista_equipos)
jornada = []
for i in range(0, len(lista_equipos), 2):
    match = (lista_equipos[i], lista_equipos[i+1])
    jornada.append(match)
#print(jornada)
return jornada

```

Aplica el algoritmo al juego de datos generado

RESPUESTA

Para aplicar la nueva función al algoritmo que hemos diseñado, simplemente llamamos a la función generadora y nos rellena una jornada, que pasamos directamente al algoritmo genético. Al cambiar los datos de entrada constantemente, el máximo de la función va a variar, porque no es el mismo para las diferentes combinaciones de partidos.

```

##### EJECUCIÓN DEL ALGORITMO GENÉTICO#####
poblacion = []
horarios = list(c_horarios.keys())
#creamos una muestra de 200 individuos aleatorios completamente, aunque más adelante pida que genere entradas aleatorias, en
#de crear_individuo_aleatorio y recibe la jornada que es la que generaremos aleatoriamente en el apartado de más adelante, y
#llamamos a la función para generar una jornada aleatoria y usamos ese nuevo valor para generar diferentes horarios para es
jornada = entrada_random()
for x in range(200):
    individuo = crear_individuo_aleatorio(jornada, horarios)
    poblacion.append(individuo)

evaluacion = evaluar_audiencia(poblacion)
mejor = max(evaluacion, key = lambda x: x[1])

#print(mejor)
#algoritmo_genetico(mejor, evaluacion)
max_generaciones = 1000
limite_sin_mejora = 500
num_generaciones_sin_mejora = 0
mejor_generacion = 0

poblacion_actual = evaluacion #la población actual es la que hemos recibido la generación aleatoria, elegimos al mejor de es
mejor_poblacion_actual = mejor #mejor es el valor que traíamos del mejor de la generación generada random
mejor_final = mejor

for generacion in range(max_generaciones):
    nueva_poblacion = algoritmo_genetico(mejor_poblacion_actual, poblacion_actual)
    poblacion_actual = evaluar_audiencia(nueva_poblacion)
    mejor_poblacion_actual = max(poblacion_actual, key=lambda x: x[1])

    if mejor_poblacion_actual[1] > mejor_final[1]:
        mejor_final = mejor_poblacion_actual
        num_generaciones_sin_mejora = 0
        mejor_generacion = generacion + 1 #guardamos como mejor generación la actual, que corresponde con el lugar que ocupa el i
    else:
        num_generaciones_sin_mejora += 1

    print(f"Generación {generacion + 1} -> con la audiencia de {mejor_poblacion_actual[1]:.3f}")

    if num_generaciones_sin_mejora >= limite_sin_mejora:
        print(f"No hay mejora en {limite_sin_mejora} generaciones, por lo que se para de ejecutar")
        break

print(f"La mejor audiencia es: {mejor_final[1]}, en la generación {mejor_generacion}")
print(f"La mejor distribución es:")
for partido, horario in mejor_final[0]:
    print(partido, "--", horario)

```

```

Generación 621 -> con la audiencia de 7.021
Generación 622 -> con la audiencia de 7.021
Generación 623 -> con la audiencia de 7.021
Generación 624 -> con la audiencia de 7.021
Generación 625 -> con la audiencia de 7.021
Generación 626 -> con la audiencia de 7.021
Generación 627 -> con la audiencia de 7.021
Generación 628 -> con la audiencia de 7.021
Generación 629 -> con la audiencia de 7.021
Generación 630 -> con la audiencia de 7.021
Generación 631 -> con la audiencia de 7.021
Generación 632 -> con la audiencia de 7.021
Generación 633 -> con la audiencia de 7.021
Generación 634 -> con la audiencia de 7.021
Generación 635 -> con la audiencia de 7.021
Generación 636 -> con la audiencia de 7.021
Generación 637 -> con la audiencia de 7.021
Generación 638 -> con la audiencia de 7.021
Generación 639 -> con la audiencia de 7.021
Generación 640 -> con la audiencia de 7.021
Generación 641 -> con la audiencia de 7.021
Generación 642 -> con la audiencia de 7.021
Generación 643 -> con la audiencia de 7.021
Generación 644 -> con la audiencia de 7.021
Generación 645 -> con la audiencia de 7.021
Generación 646 -> con la audiencia de 7.021
Generación 647 -> con la audiencia de 7.021
Generación 648 -> con la audiencia de 7.021
Generación 649 -> con la audiencia de 7.021
Generación 650 -> con la audiencia de 7.021
Generación 651 -> con la audiencia de 7.021
No hay mejora en 500 generaciones, por lo que se para de ejecutar
La mejor audiencia es: 7.0205, en la generación 151
La mejor distribución es:
('Atlético de Madrid', 'CA Osasuna') -- D18
('Levante UD', 'Elche CF') -- L20
('Deportivo Alavés', 'Rayo Vallecano') -- S12
('Real Betis', 'Athletic Club') -- S18
('Real Sociedad', 'RCD Espanyol') -- S20
('Getafe CF', 'Villarreal CF') -- S20
('FC Barcelona', 'Real Madrid') -- D20
('RC Deportivo', 'Celta') -- S16
('Valencia CF', 'Racing de Santander') -- V20
('Sevilla FC', 'Málaga CF') -- D16

```

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

RESPUESTA

- <https://docs.python.org/es/3/library/itertools.html>
- Apuntes de la asignatura Algoritmos de Optimización
- Apuntes de la asignatura de Python para la Inteligencia Artificial

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

RESPUESTA

El siguiente paso en este problema, sería cambiar el enfoque. Como usted comentó en la materia, podríamos elegir entre generar una jornada o un calendario completo. Nos hemos centrado en encontrar para una jornada la mejor combinación, es decir el máximo de espectadores, pero el problema se dimensiona cuando lo pretendemos hacer para toda la jornada de liga, donde se nos podrían plantear dos posibles soluciones: o bien plantear el algoritmo de manera independiente a cada jornada (es decir nuestro planteamiento nos valdría) y posteriormente sumar todas las audiencias de cada uno; o directamente representar cada individuo como la liga completa con sus 38 jornadas, y las restricciones posibles de el calendario y de las normas básicas de una liga de fútbol. El tamaño por supuesto se incrementaría así como la complejidad de tratarlo.

Otras posibles mejoras que se han explorado han sido la de aumentar su población, de 200 a más individuos (fue implementado y testeado con 100, y luego aumentamos) aún, la de aumentar el % de mutaciones, pero nos hemos quedado con estos valores por su sencillez y por lo que creemos su cercanía a una solución óptima. Por ejemplo, si que se han ajustado que en un principio no hicimos que padre/madre sean diferentes, y que las mutaciones no dieran el hijo idéntico, ya que podía pasar, y estos cambios fueron decisivos para ver mayores variaciones a lo largo de varias generaciones, aunque sigue influyendo mucho la jornada utilizada como la propia aleatoriedad que aporta el algoritmo. Esto se puede apreciar en las 2 ejecuciones diferentes del algoritmo genético, con una prefijada o con una completamente aleatoria, donde en esta última en algunas ocasiones las variaciones se alargan hasta la generación 7xx, mientras que en la prefijada no suele pasar de 50.

